Week 5 : Recursion ,Sorting Objectives

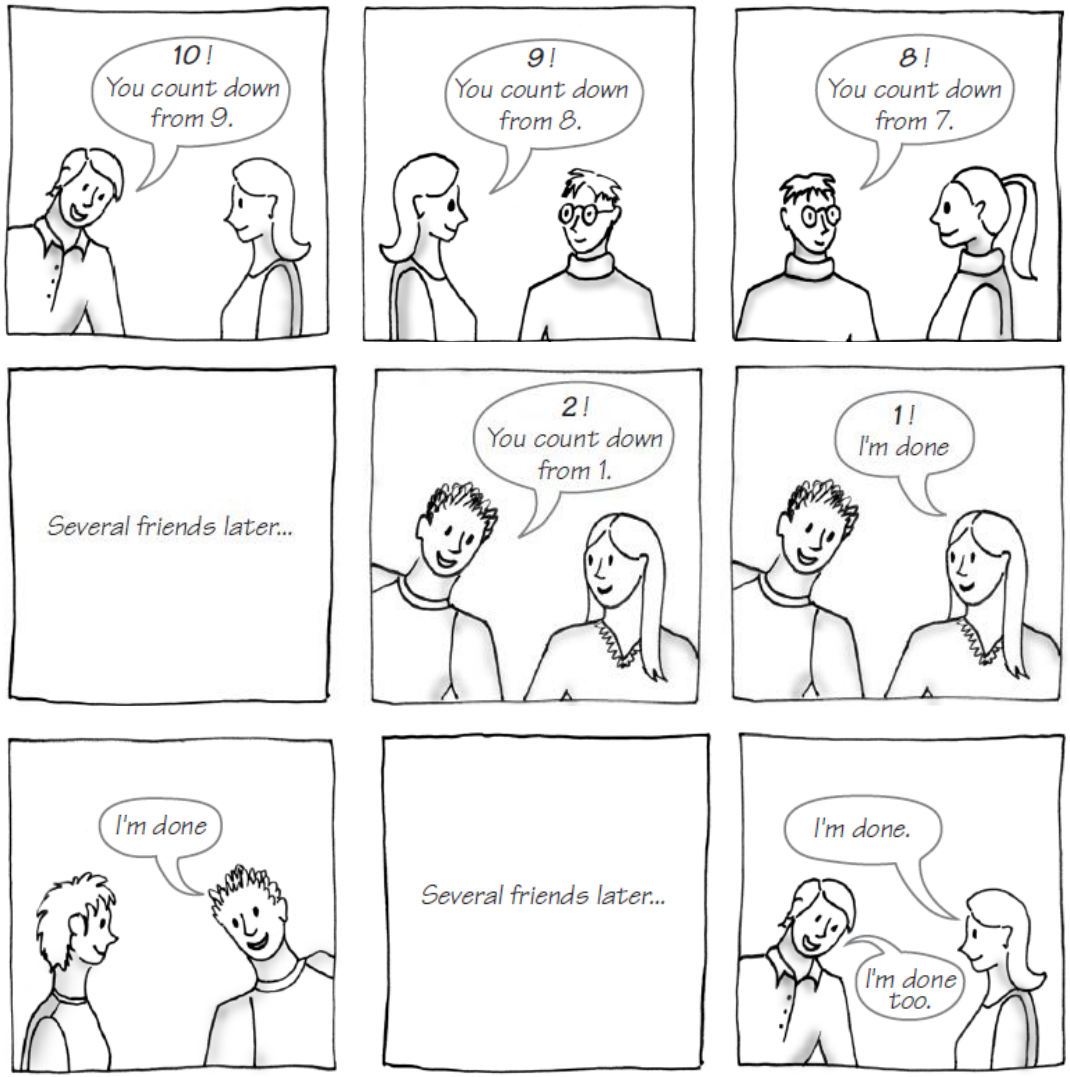
Upon completion of this unit, students will be able to understand and explain the following sub topics :

- Recursion
- Sorting: Selection sorting, insertion sorting
- Merge Sort

What Is Recursion?

- Consider hiring a contractor to build
  - He hires a subcontractor for a portion of the job
  - That subcontractor hires a sub-subcontractor to do a smaller portion of job
- The last sub-sub- … subcontractor finishes
  - Each one finishes and reports “done” up the line

Exampl



- FIGURE 9-1 Counting down from 10

Example: The Countdown

- Recursive Java method to do `countDown`.

```
/** Counts down from a given positive integer.
    @param integer  An integer > 0.
*/
public static void countDown(int integer)
{
   System.out.println(integer);
   if (integer > 1)
      countDown(integer - 1);
} // end countDown
```

Definition

- Recursion is a problem-solving process
  - Breaks a problem into identical but smaller problems.
- A method that calls itself is a recursive method.
  - The invocation is a recursive call or recursive invocation.

Design Guidelines

- Method must be given an input value
- Method definition must contain logic that involves this input, leads to different cases
- One or more cases should provide solution that does not require recursion
  - Else infinite recursion
- One or more cases must include a recursive invocation

Programming Tip

- Iterative method contains a loop
- Recursive method calls itself
- Some recursive methods contain a loop and call themselves
  - If the recursive method with loop uses while, make sure you did not mean to use an if statement

Tracing a Recursive Method

- The effect of the method call `countDown(3)`

`countDown(3)`

Display 3
Call `countDown(2)`

`countDown(2)`

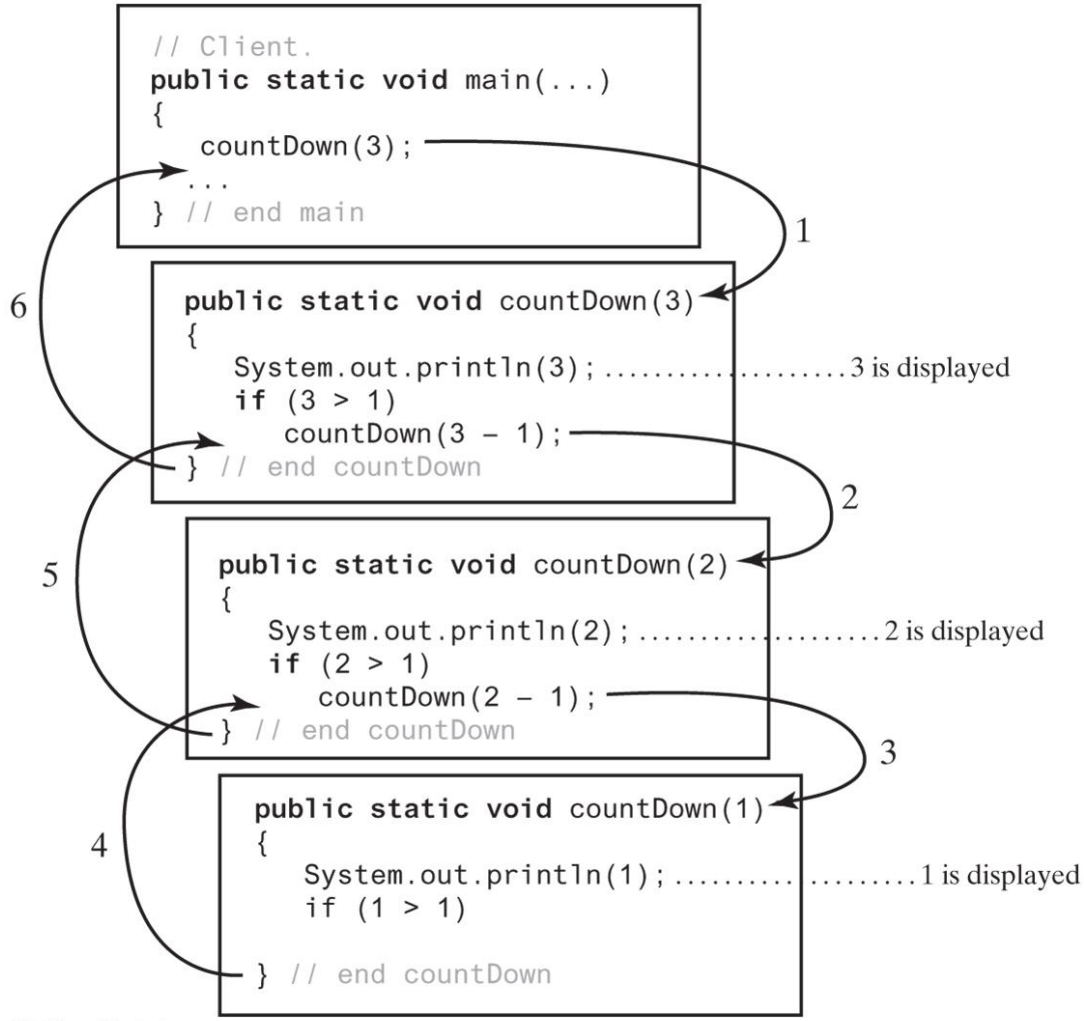
Display 2
Call `countDown(1)`

`countDown(1)`

Display 1

Tracing a Recursive Method

- Tracing the execution of `countDown(3)`

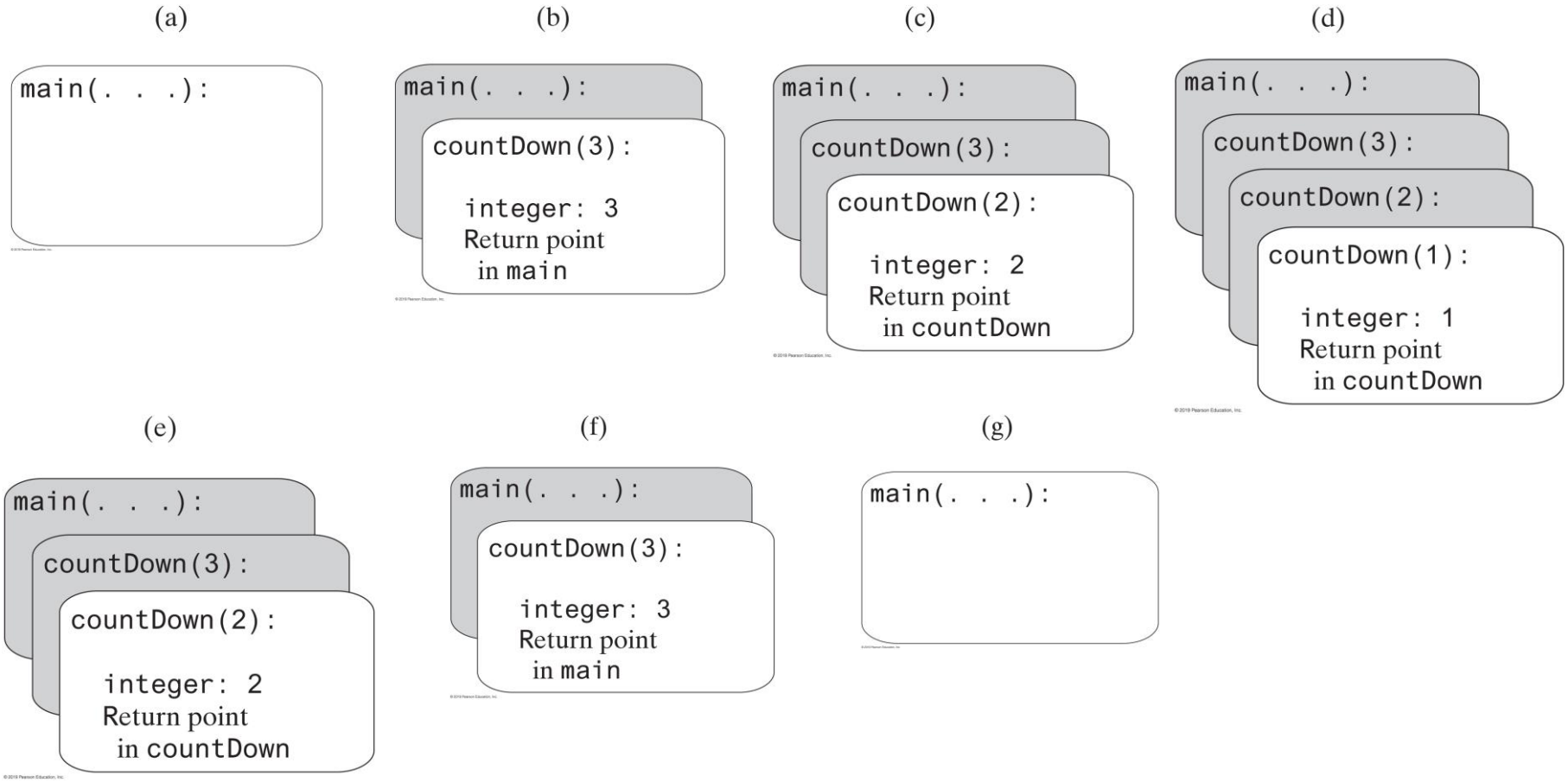


© 2019 Pearson Education, Inc.

Stack of Activation Records

- Each call to a method generates an activation record
- Recursive method uses more memory than an iterative method
  - Each recursive call generates an activation record
- If recursive call generates too many activation records, could cause stack overflow

Stack of Activation Records



- The stack of activation records during the execution of the call `countDown(3)`

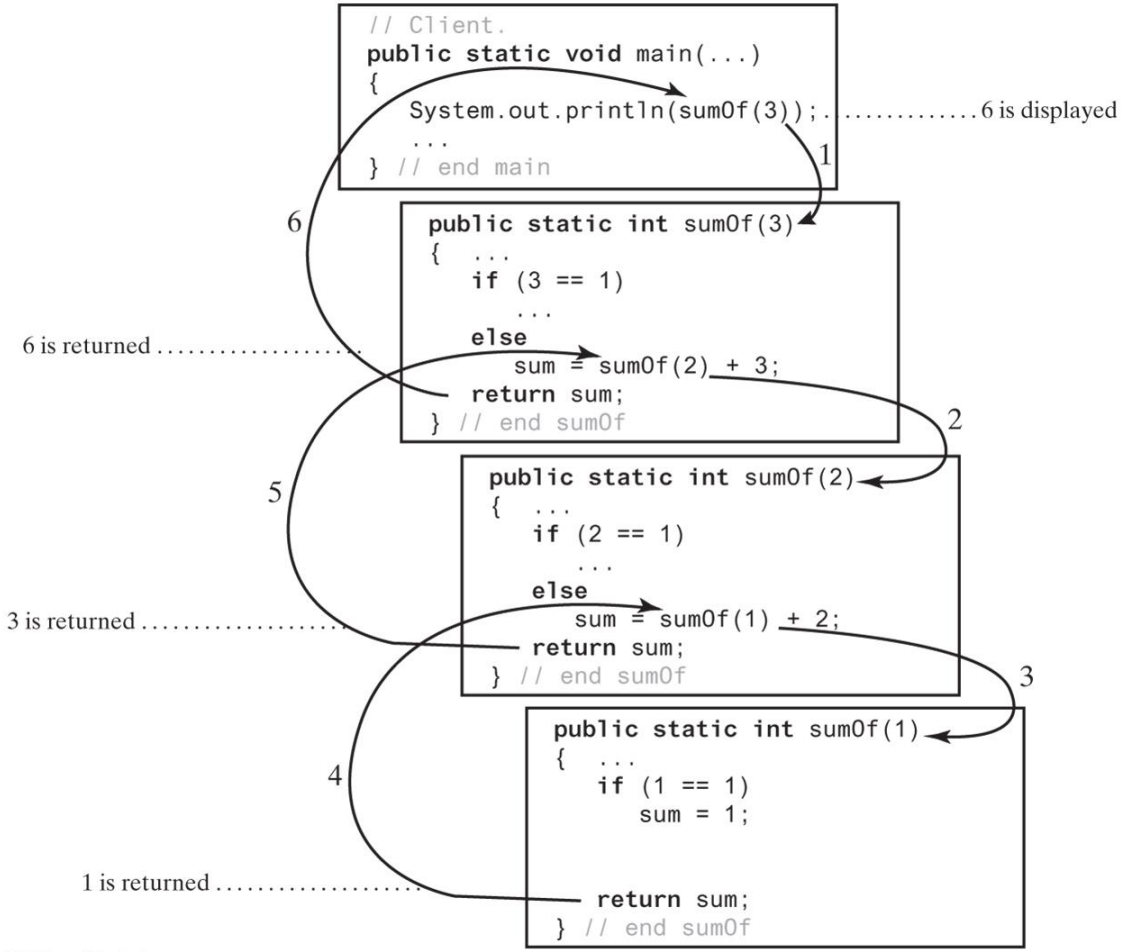
Recursive Methods That Return a Value

- Recursive method to calculate

```
/** @param n  An integer > 0.
    @return  The sum 1 + 2 + ... + n. */
public static int sumOf(int n)
{
   int sum;
   if (n == 1)
      sum = 1;                // Base case
   else
      sum = sumOf(n - 1) + n; // Recursive call

   return sum;
} // end sumOf
```

$$\sum_{i=1}^{n} i$$

Tracing the execution of sumOf(3)



© 2019 Pearson Education, Inc.

Recursively Processing an Array

- Given definition of a recursive method to display array.

```
/** Displays the integers in an array.
   @param array  An array of integers.
   @param first  The index of the first integer displayed.
   @param last   The index of the last integer displayed,
                 0 <= first <= last < array.length. */
public static void displayArray(int[] array, int first, int last)
```

Recursively Processing an Array

- Starting with `array[first]`

```
public static void displayArray(int array[], int first, int last)
{
   System.out.print(array[first] + " ");
   if (first < last)
      displayArray(array, first + 1, last);
} // end displayArray
```

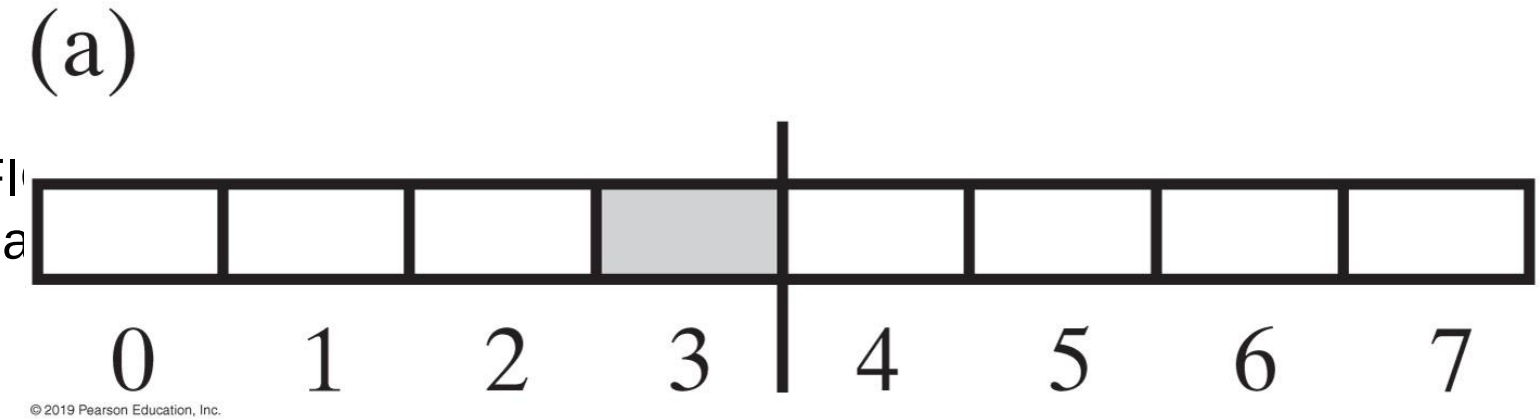
Recursively Processing an Array

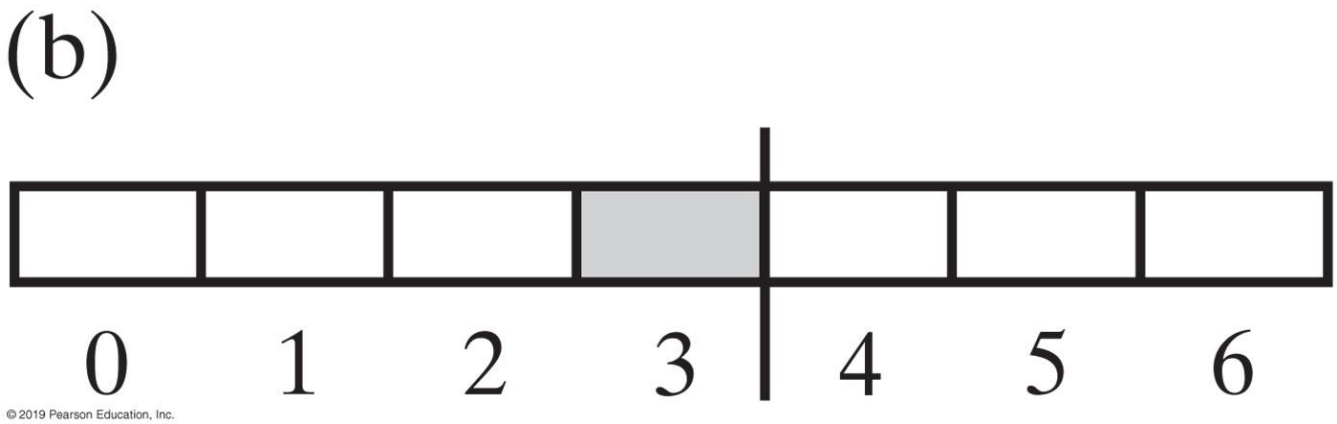
- Starting with `array[last]`

```
public static void displayArray(int array[], int first, int last)
{
   if (first <= last)
   {
      displayArray(array, first, last - 1);
      System.out.print(array[last] + " ");
   } // end if
} // end displayArray
```

Recursively Processing an Array

- Fl
ha





Recursively Processing an Array

```
public static void displayArray(int array[], int first, int last)
{
   if (first == last)
      System.out.print(array[first] + " ");
   else
   {
      int mid = (first + (last - first) / 2;
      displayArray(array, first, mid);
      displayArray(array, mid + 1, last);
   } // end if
} // end displayArray
```

Why?

Displaying a Bag

- Recursive method that is part of an implementation of an ADT often is `private`.

```
public void display()
{
   displayArray(0, numberOfEntries - 1);
} // end display

private void displayArray(int first, int last)
{
   System.out.println(bag[first]);
   if (first < last)
      displayArray(first + 1, last);
} // end displayArray
```

Recursively Processing a Linked Chain

- Display data in first node and recursively display data in rest of chain.

```
public void display()
{
   displayChain(firstNode);
} // end display

private void displayChain(Node nodeOne)
{
   if (nodeOne != null)
   {
      System.out.println(nodeOne.getData()); // Display data in first node
      displayChain(nodeOne.getNextNode());   // Display rest of chain
   } // end if
} // end displayChain
```

Recursively Processing a Linked Chain

```
public void displayBackward()
{
   displayChainBackward(firstNode);
} // end displayBackward

private void displayChainBackward(Node nodeOne)
{
   if (nodeOne != null)
   {
      displayChainBackward(nodeOne.getNextNode());
      System.out.println(nodeOne.getData());
   } // end if
} // end displayChainBackward
```

- Displaying a chain backwards. Traversing chain of linked nodes in reverse order easier when done recursively.

Time Efficiency of Recursive Methods

- Using proof by induction, we conclude method is O(n).

```
public static void countDown(int n)
{
   System.out.println(n);
   if (n > 1)
      countDown(n - 1);
} // end countDown
```

Time Efficiency of Computing x^n

- Efficiency of algorithm is O(log n)

$$x^n = (x^{n/2})^2 \text{ when } n \text{ is even and positive}$$

$$x^n = x\,(x^{(n-1)/2})^2 \text{ when } n \text{ is odd and positive}$$

$$x^0 = 1$$

Tail Recursion

- When the last action performed by a recursive method is a recursive call.
- In a tail-recursive method, the last action is a recursive call
- This call performs a repetition that can be done by using iteration.
- Converting a tail-recursive method to an iterative one is usually a straightforward process.

```
public static void countDown(int n)
{
   System.out.println(n);
   if (n > 1)
      countDown(n - 1);
} // end countDown
```

Using a Stack Instead of Recursion

- Converting a recursive method to an iterative one

```
public static void countDown(int integer)
{
   if (integer >= 1)
   {
      System.out.println(integer);
      countDown(integer - 1);
   } // end if
} // end countDown
```

- An iterative version

```
public static void countDown(int integer)
{
   while (integer >= 1)
   {
      System.out.println(integer);
      integer = integer − 1;
   } // end while
} // end countDown
```

Using a Stack Instead of Recursion

- An iterative displayArray to maintain its own stack

```
public void displayArray(int first, int last)
{
   boolean done = false;
   StackInterface<Record> programStack = new LinkedStack<>();
   programStack.push(new Record(first, last));
   while (!done && !programStack.isEmpty())
   {
      Record topRecord = programStack.pop();
      first = topRecord.first;
      last = topRecord.last;

      if (first == last)
         System.out.println(array[first] + " ");
      else
      {
         int mid = first + (last - first) / 2;
         // Note the order of the records pushed onto the stack
         programStack.push(new Record(mid + 1, last));
         programStack.push(new Record(first, mid));
      } // end if
   } // end while
} // end displayArray
```

Sorting

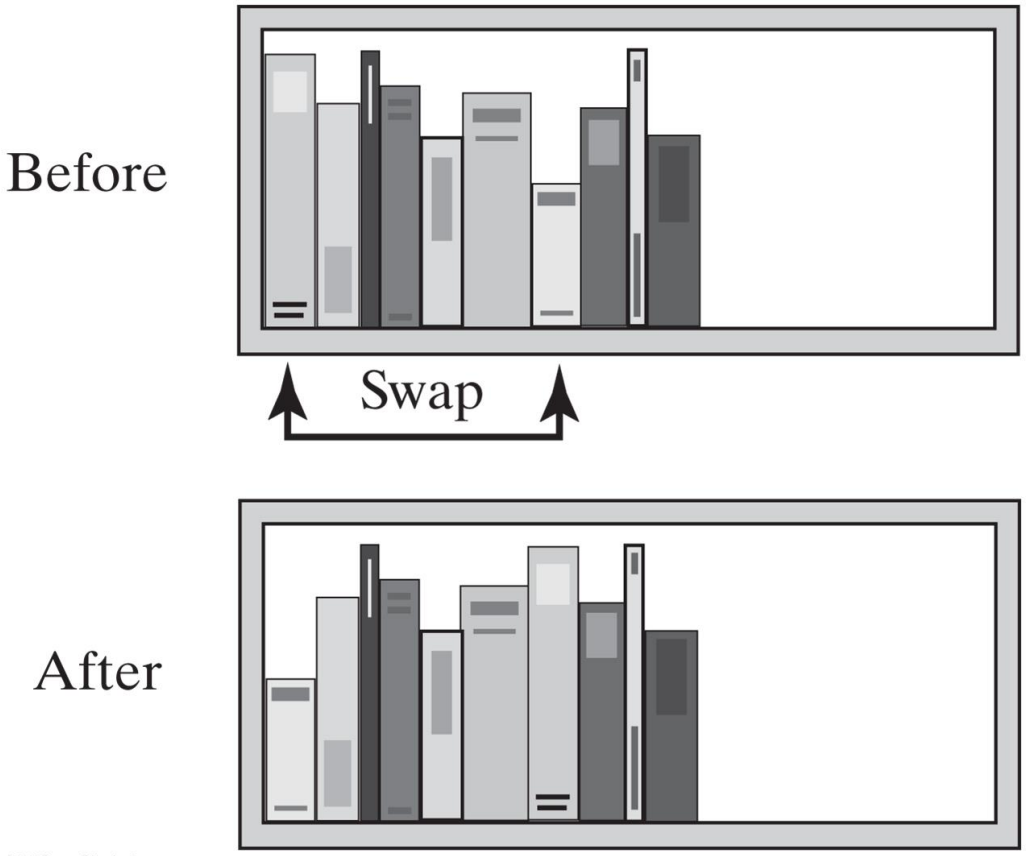
- We seek algorithms to arrange items, a_i such that:

$$\text{entry } 1 \leq \text{entry } 2 \leq \ldots \leq \text{entry } n$$

- Sorting an array is usually easier than sorting a chain of linked nodes
- Efficiency of a sorting algorithm is significant

Selection Sort

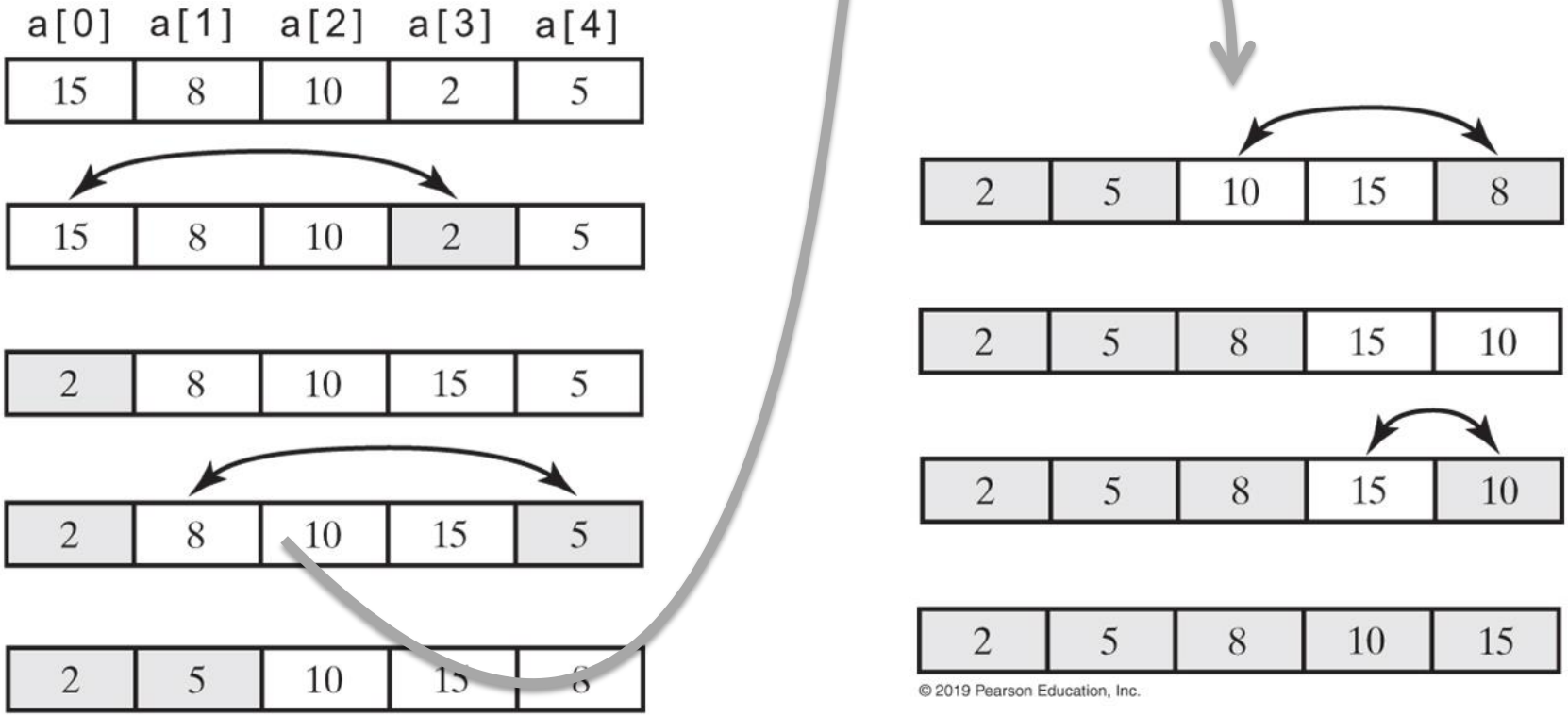
- Before and after exchanging the shortest book and the first book



© 2019 Pearson Education, Inc.

Selection Sort

- A selection sort of an array of integers into ascending order



© 2019 Pearson Education, Inc.

Iterative Selection Sort

- This pseudocode describes an iterative algorithm for the selection sort

```
Algorithm selectionSort(a, n)
// Sorts the first n entries of an array a.
for (index = 0; index < n − 1; index++)
{
        indexOfNextSmallest = the index of the smallest value among
                                                a[index], a[index + 1], . . . , a[n − 1]
        Interchange the values of a[index] and a[indexOfNextSmallest]
        // Assertion: a[0] ≤ a[1] ≤ . . . ≤ a[index], and these are the smallest
        //of the original array entries. The remaining array entries begin at a[index + 1].
}
```

Iterative Selection Sort (Part 1)

- A class for sorting an array using selection sort

```
/**  A class of static, iterative methods for sorting an array of
   Comparable objects from smallest to largest.  */
public class SortArray
{
   /** Sorts the first n objects in an array into ascending order.
       @param a  An array of Comparable objects.
       @param n  An integer > 0. */
   public static <T extends Comparable<? super T>>
         void selectionSort(T[] a, int n)
   {
      for (int index = 0; index < n - 1; index++)
      {
         int indexOfNextSmallest = getIndexOfSmallest(a, index, n - 1);
         swap(a, index, indexOfNextSmallest);
      // Assertion: a[0] <= a[1] <= . . . <= a[index] <= all other a[i]
      } // end for
   } // end selectionSort
```

Iterative Selection Sort (Part 2)

```
// Finds the index of the smallest value in a portion of an array a.
// Precondition: a.length > last >= first >= 0.
// Returns the index of the smallest value among
// a[first], a[first + 1], . . . , a[last].
private static <T extends Comparable<? super T>>
        int getIndexOfSmallest(T[] a, int first, int last)
{
   T min = a[first];
   int indexOfMin = first;
   for (int index = first + 1; index <= last; index++)
   {
      if (a[index].compareTo(min) < 0)
      {
         min = a[index];
         indexOfMin = index;
      } // end if
      // Assertion: min is the smallest of a[first] through a[index].
   } // end for

   return indexOfMin;
} // end getIndexOfSmallest
```

- A class for sorting an array using selection sort

Iterative Selection Sort (Part 3)

- A class for sorting an array using selection sort

```
  // Swaps the array entries a[i] and a[j].
   private static void swap(Object[] a, int i, int j)
   {
      Object temp = a[i];
      a[i] = a[j];
      a[j] = temp;
   } // end swap
} // end SortArray
```

Recursive Selection Sort

- Recursive selection sort algorithm

Algorithm **selectionSort(a, first, last)**

// *Sorts the array entries* a[first] *through* a[last] *recursively.*

if (first < last)

{

indexOfNextSmallest = *the index of the smallest value among*

a[first], a[first + 1], . . . , a[last]

Interchange the values of a[first] *and* a[indexOfNextSmallest]

// *Assertion:* a[0] ≤ a[1] ≤ . . . ≤ a[first] *and these are the smallest*

// *of the original array entries. The remaining array entries begin at* a[first + 1].

selectionSort(a, first + 1, last)

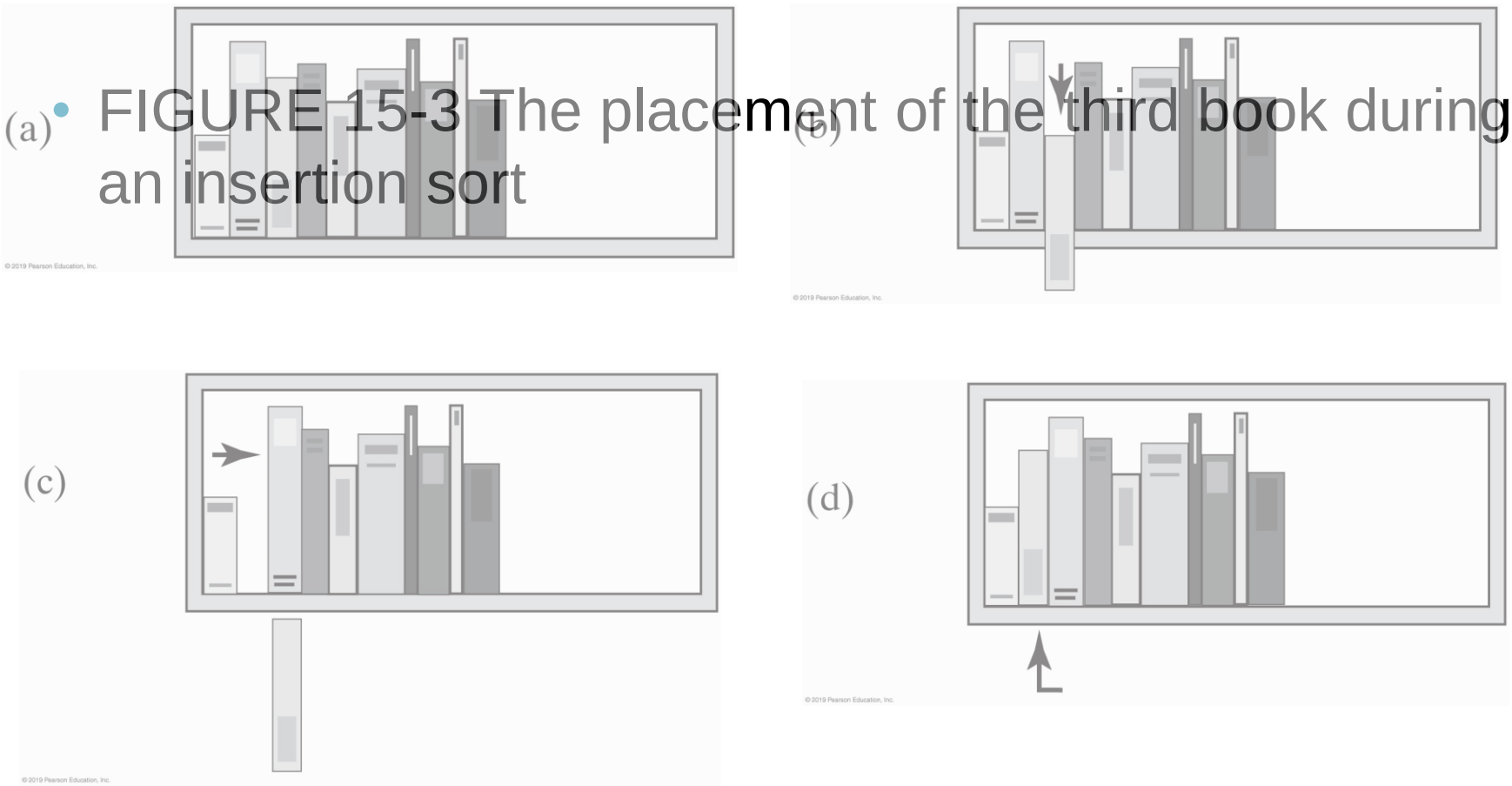
}

Efficiency of Selection Sort

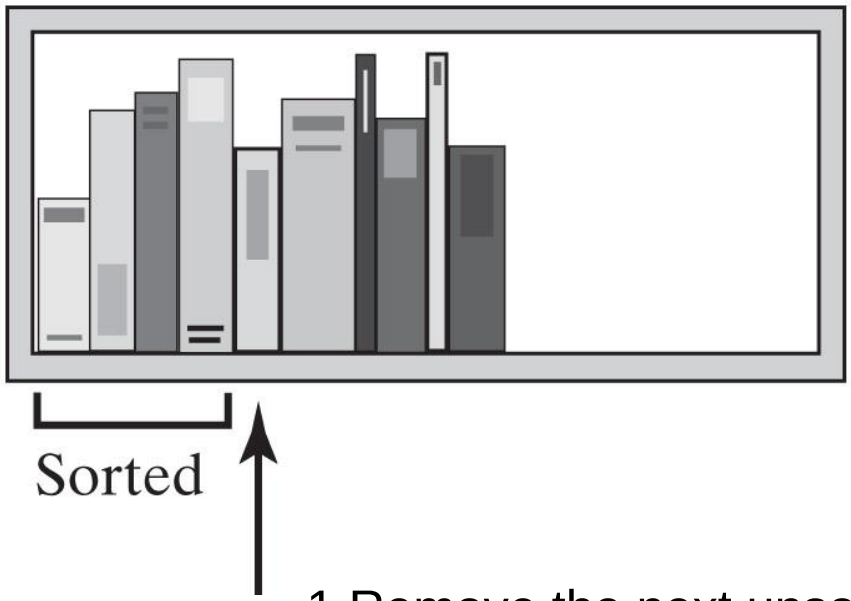
- Selection sort is $O(n^2)$ regardless of the initial order of the entries.
  - Requires $O(n^2)$ comparisons
  - Does only $O(n)$ swaps

Insertion Sort

- FIGURE 15-3 The placement of the third book during an insertion sort

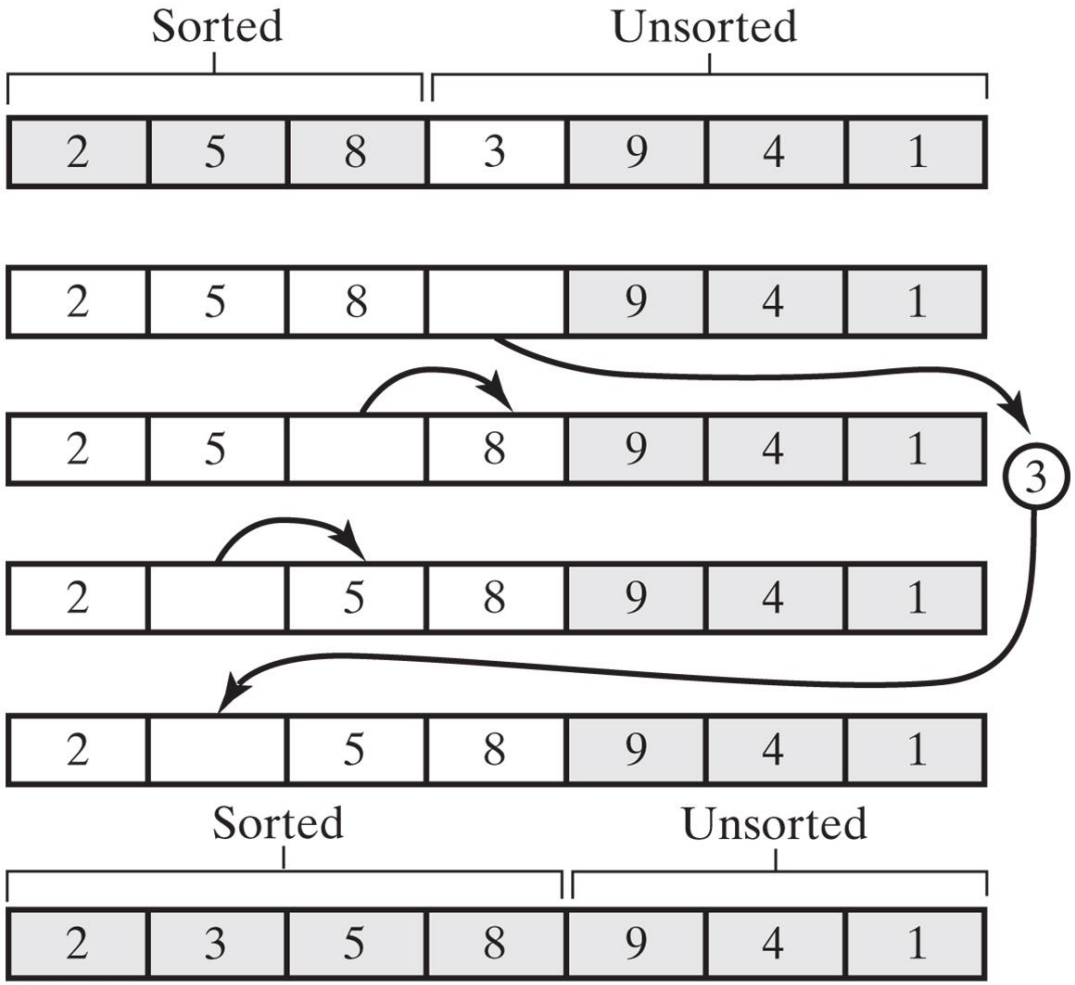


Insertion Sort



1. Remove the next unsorted book.
2. Slide the sorted books to the right one by one until you find the right spot for the removed book.
3. Insert the book into its new position

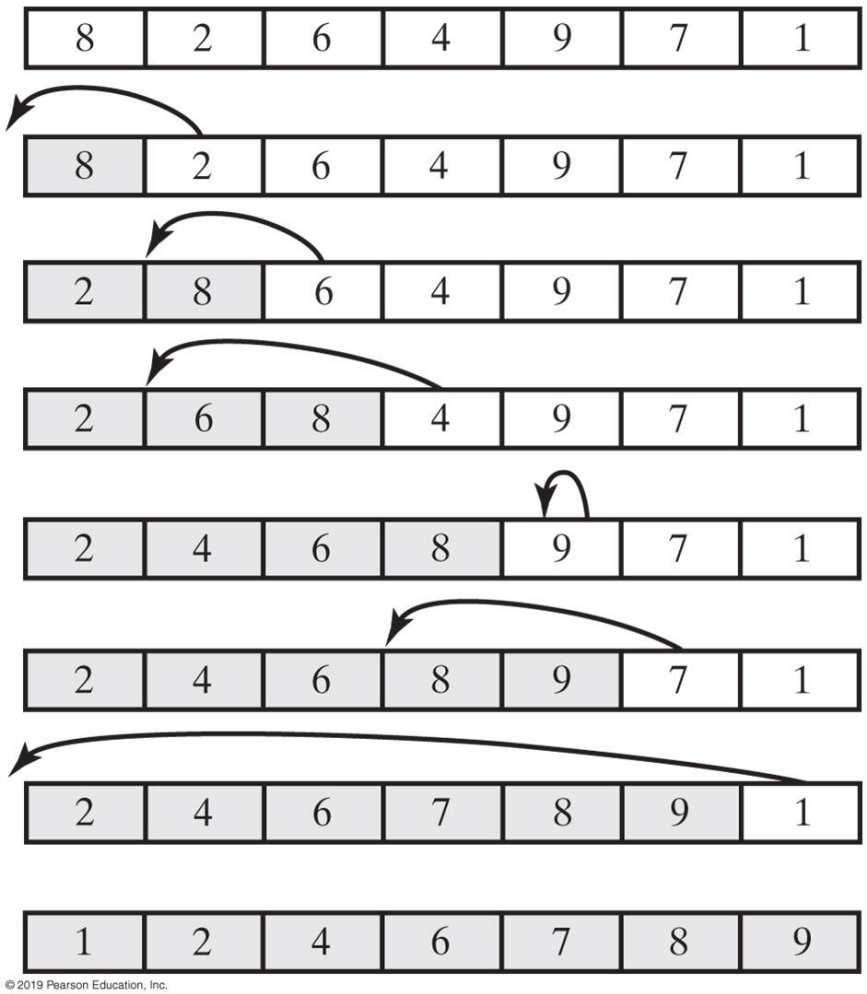
Insertion Sort



© 2019 Pearson Education, Inc.

- Inserting the next unsorted entry into its proper location within the sorted portion of an array during an insertion sort

Insertion Sort



© 2019 Pearson Education, Inc.

- An insertion sort of an array of integers into ascending order

Iterative Insertion Sort

- Iterative algorithm describes an insertion sort of the entries at indices first through last of the array `a`

Algorithm **insertionSort(a, first, last)**

// *Sorts the array entries* a[first] *through* a[last] *iteratively*.

for (unsorted = first + 1 through last)

{

nextToInsert = a[unsorted] insertInOrder(nextToInsert, a, first, unsorted −

}

Iterative Insertion Sort

```
Algorithm insertInOrder(anEntry, a, begin, end)
// Inserts anEntry into the sorted entries a[begin] through a[end].
index = end                    //Index of last entry in the sorted portion
// Make room, if needed, in sorted portion for another entry
while ( (index >= begin) and (anEntry < a[index]) )
{
        a[index + 1] = a[index] // Make room
        index--
}
// Assertion: a[index + 1] is available.
a[index + 1] = anEntry         //Insert
```

- Pseudocode of method, `insertInOrder`, to perform the insertions.

Recursive Insertion Sort

- This pseudocode describes a recursive insertion sort.

```
Algorithm insertionSort(a, first, last)
// Sorts the array entries a[first] through a[last] recursively.
if (the array contains more than one entry)
{
        Sort the array entries a[first] through a[last − 1]
        Insert the last entry a[last] into its correct sorted position within the rest of the arra
}
```

Recursive Insertion Sort

```
public static <T extends Comparable<? super T>>
      void insertionSort(T[] a, int first, int last)
{
   if (first < last)
   {
      // Sort all but the last entry
      insertionSort(a, first, last - 1);

      // Insert the last entry in sorted order
      insertInOrder(a[last], a, first, last - 1);
   } // end if
} // end insertionSort
```

- Implementing the algorithm in Java

Recursive Insertion Sort

- First draft of `insertInOrder` algorithm.

Algorithm **insertInOrder(anEntry, a, begin, end)**

// *Inserts* anEntry *into the sorted array entries* a[begin] *through* a[end].

//*First draft.*

if (anEntry >= a[end])

```
        a[end + 1] =  anEntry
```

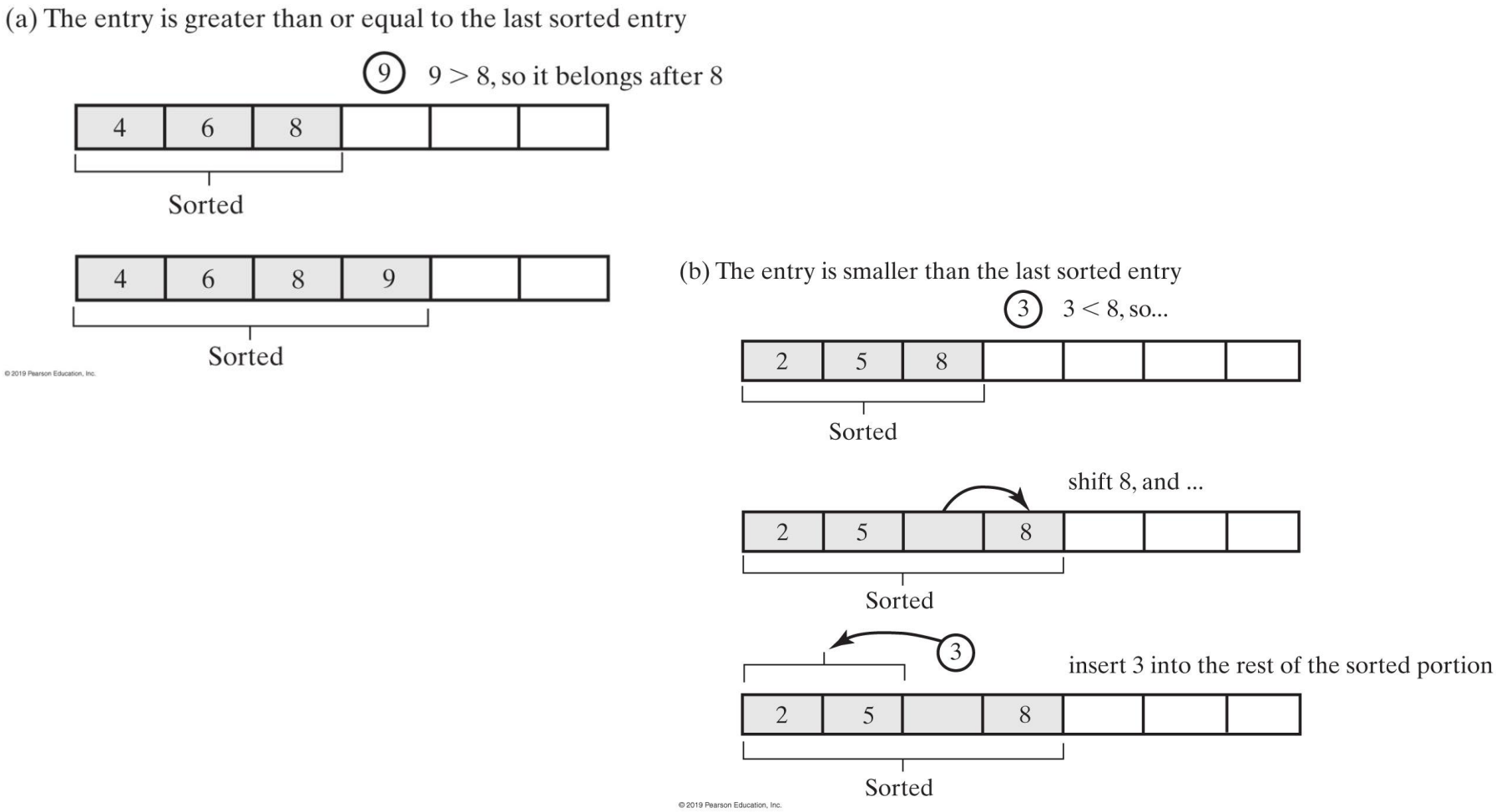
else

{

```
        a[end  +  1]  =  a[end]  insertInOrder(anEntry, a, begin, end − 1)
```

}

Insertion Sort



- Inserting the first unsorted entry into the sorted portion of the array

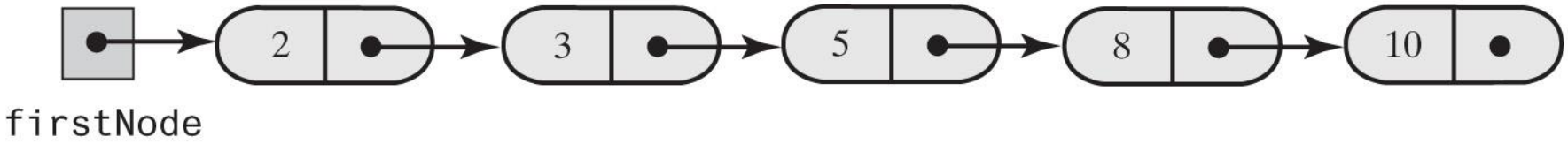
Recursive Insertion Sort

```
Algorithm insertInOrder(anEntry, a, begin, end)
// Inserts anEntry into the sorted array entries a[begin] through a[end].
//Revised draft.
if (anEntry >= a[end])
        a[end + 1] =  anEntry
else if (begin < end)
{
        a[end  +  1]  =  a[end]  insertInOrder(anEntry, a, begin, end − 1)
}
else // begin == end and anEntry < a[end]
{
        a[end + 1] =  a[end]
        a[end] = anEntry
}
```

- The algorithm `insertInOrder`: final draft.
  Note: insertion sort efficiency (worst case) is $O(n^2)$

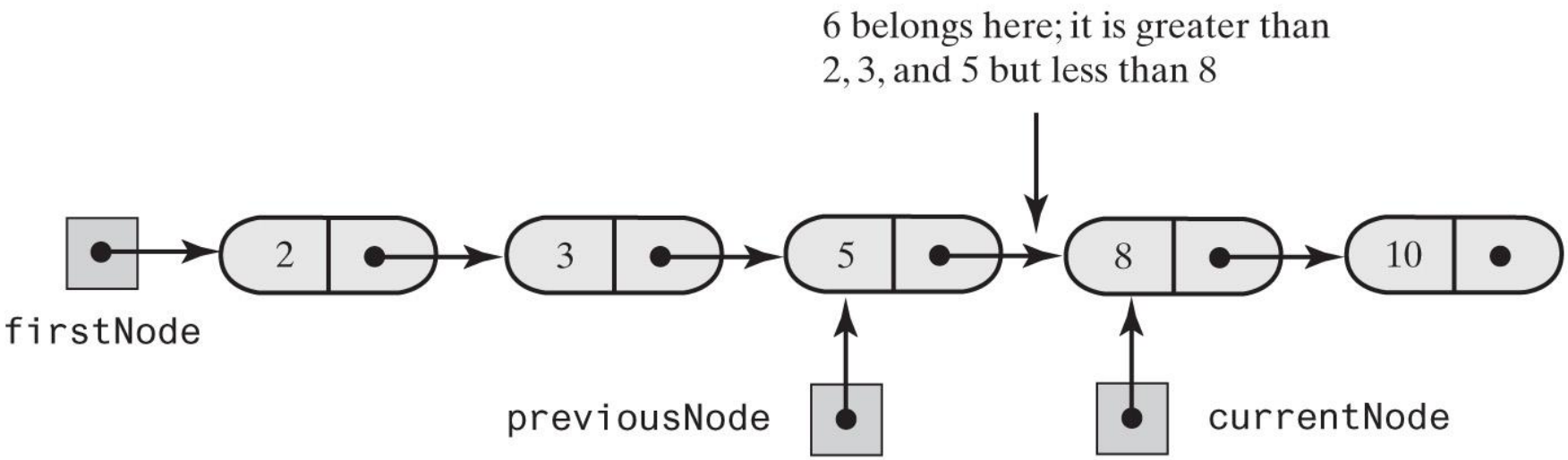
Insertion Sort with a Linked Chain

- FIGURE 15-8 A chain of integers sorted into ascending order



© 2019 Pearson Education, Inc.

Insertion Sort with a Linked Chain

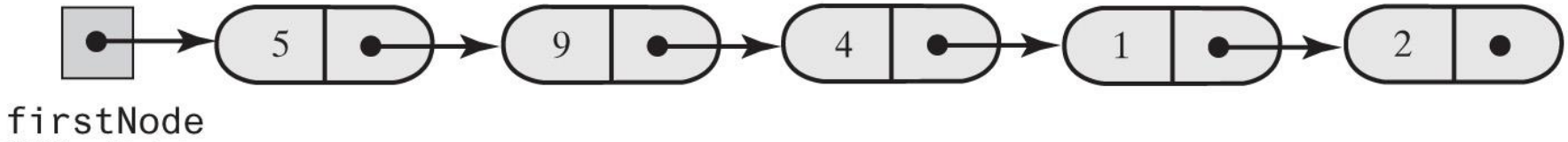


© 2019 Pearson Education, Inc.

- FIGURE 15-9 During the traversal of a chain to locate the insertion point, save a reference to the node before the current one

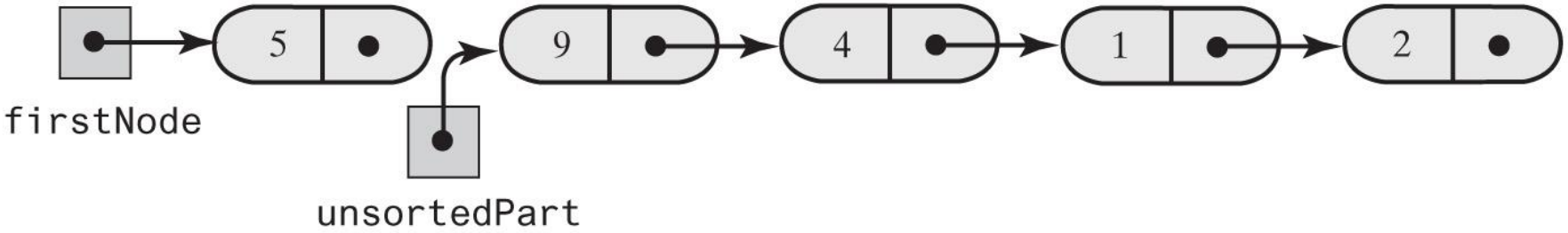
Insertion Sort with a Linked Chain

(a) The original chain



© 2019 Pearson Education, Inc.

(b) The two pieces



© 2019 Pearson Education, Inc.

Insertion Sort with a Linked Chain

```
public class LinkedGroup<T extends Comparable<? super T>>
{
   private Node firstNode;
   int length; // Number of objects in the group

   //  . . .
       private class Node
       {
          //  private inner class Node is implemented here.
        }
```

- Add a sort method to a class `LinkedGroup` that uses a linked chain to represent a certain collection

Insertion Sort with a Linked Chain

```
private void insertInOrder(Node nodeToInsert)
  {
    T item = nodeToInsert.getData();
    Node currentNode = firstNode;
    Node previousNode = null;

    // Locate insertion point
    while ( (currentNode != null) &&
            (item.compareTo(currentNode.getData()) > 0) )
    {
      previousNode = currentNode;
      currentNode = currentNode.getNextNode();
    } // end while

    // Make the insertion
    if (previousNode != null)
    {  // Insert between previousNode and currentNode
      previousNode.setNextNode(nodeToInsert);
      nodeToInsert.setNextNode(currentNode);
    }
    else // Insert at beginning
    {
      nodeToInsert.setNextNode(firstNode);
      firstNode = nodeToInsert;
    } // end if
  } // end insertInOrder
```

Class has an inner class `Node` with `set` and `get` methods

Insertion Sort with a Linked Chain

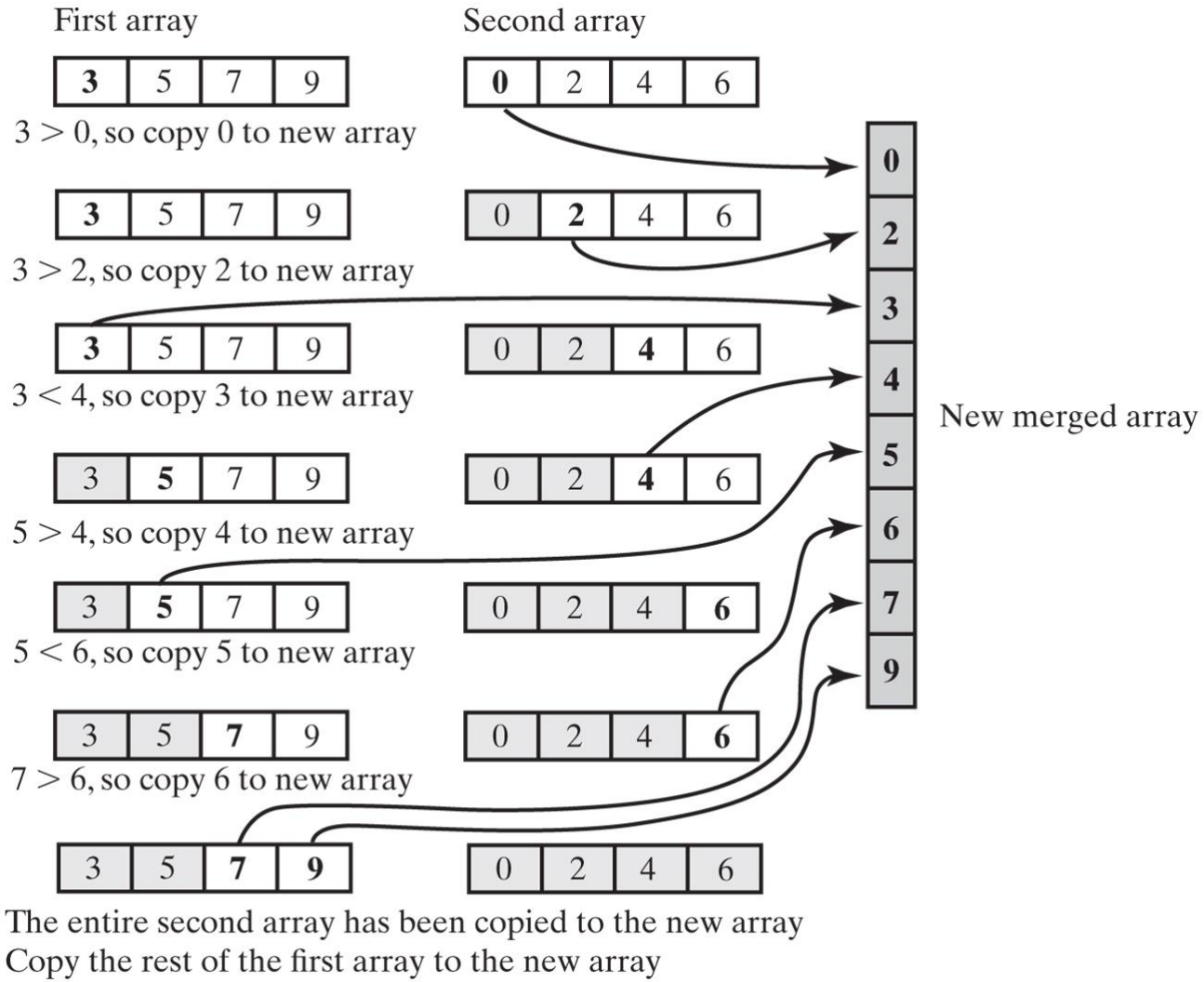
- Insertion sort method

```
public void insertionSort()
{
   // If fewer than two items are in the list, there is nothing to do
   if (length > 1)
   {
      // Assertion: firstNode != null

      // Break chain into 2 pieces: sorted and unsorted
      Node unsortedPart = firstNode.getNextNode();
      // Assertion: unsortedPart != null
      firstNode.setNextNode(null);

      while (unsortedPart != null)
      {
         Node nodeToInsert = unsortedPart;
         unsortedPart = unsortedPart.getNextNode();
         insertInOrder(nodeToInsert);
      } // end while
   } // end if
} // end insertionSort
```

Merge Sort

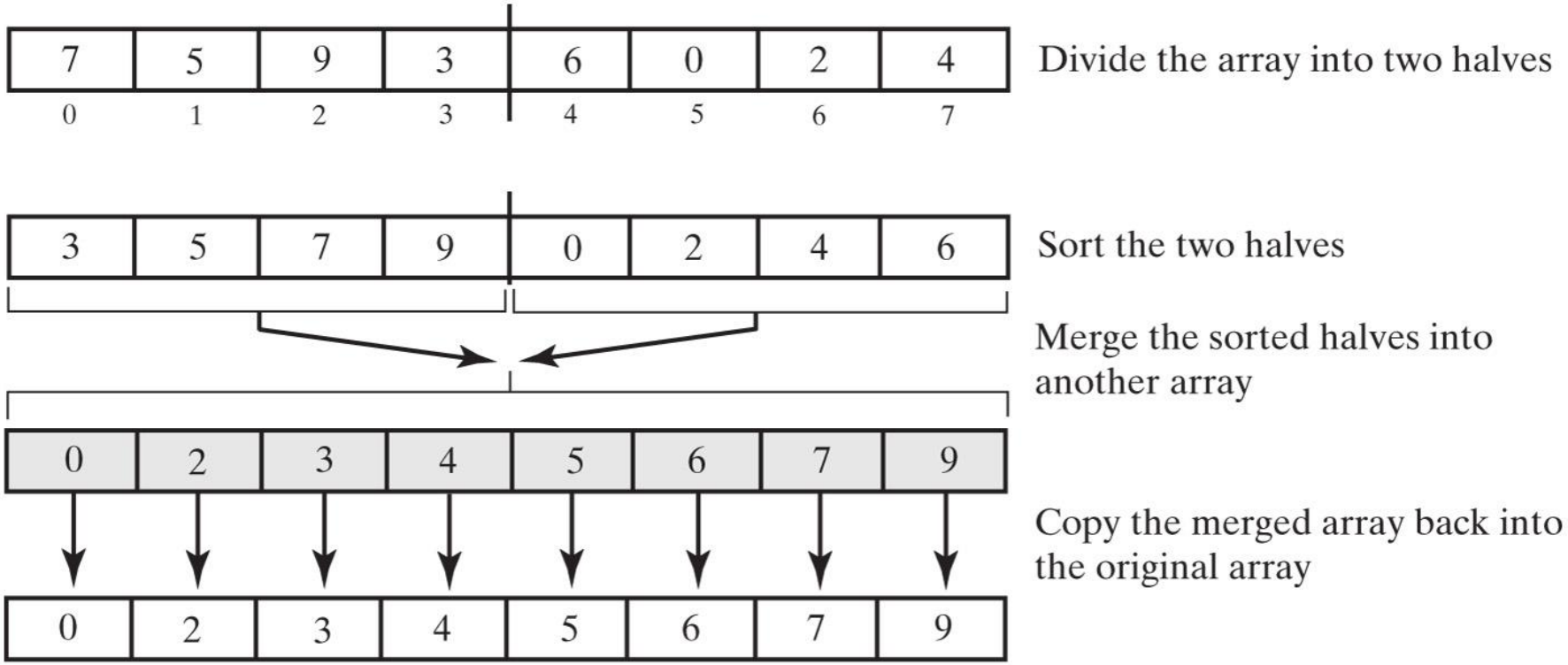
- Divides an array into halves
- Sorts the two halves,
  - Then merges them into one sorted array.
- The algorithm for merge sort is usually stated recursively.
- Major programming effort is in the merge process

Merge Sort



- FIGURE 10–1 Merging two sorted arrays into one sorted array

Marge Sort



© 2019 Pearson Education, Inc.

Recursive Merge Sort

- Recursive algorithm for merge sort.

Algorithm mergeSort(a, tempArray, **first**, last)

// *Sorts the array entries* a[first..last] *recursively.*

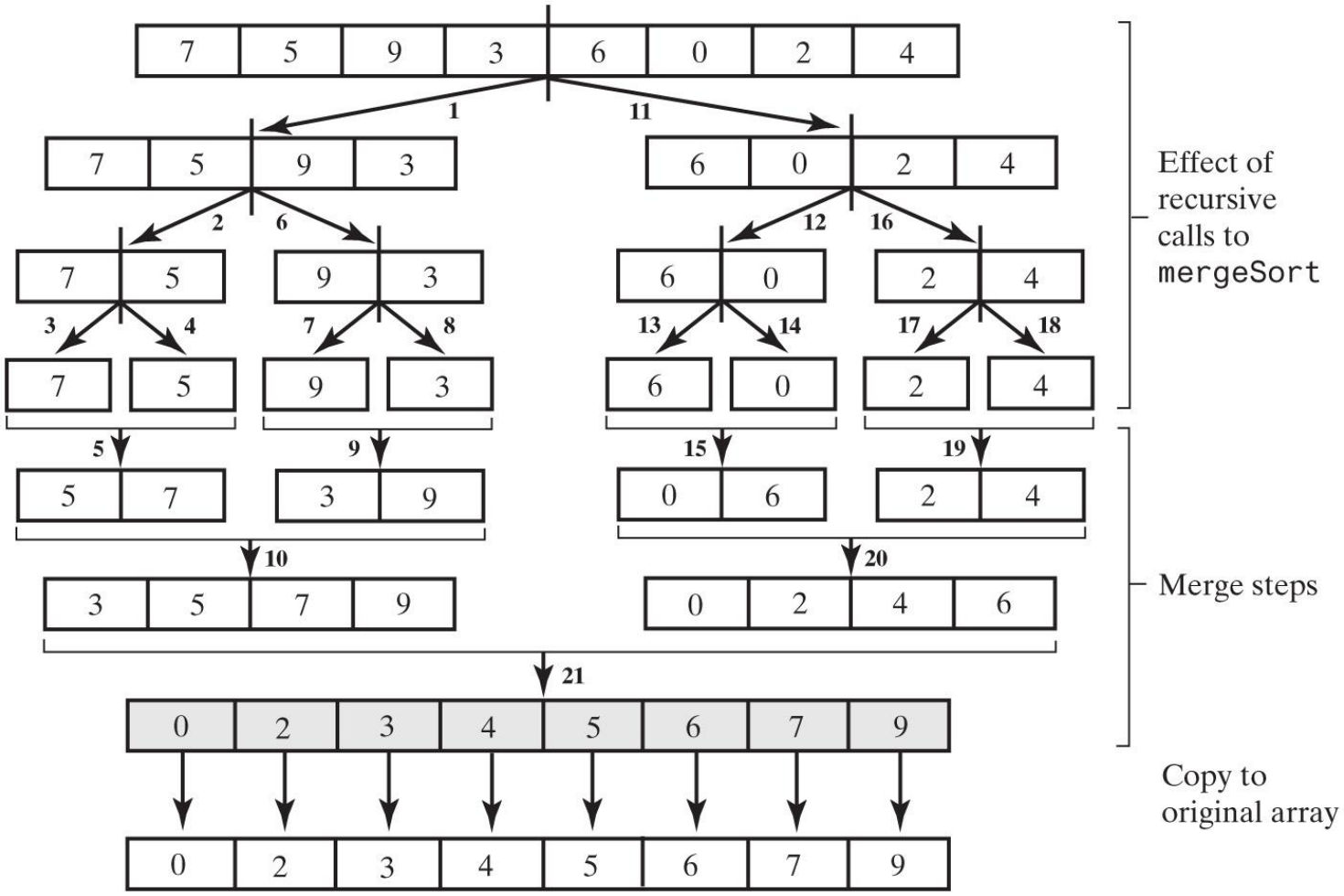
```
if (first < last)
{
        mid = approximate midpoint between first and last
        mergeSort(a, tempArray, first, mid)
        mergeSort(a, tempArray, mid + 1, last)
        Merge the sorted halves a[first..mid] and a[mid + 1..last] using the array tempA
}
```

Recursive Merge Sort

```
Algorithm merge(a, tempArray, first, mid, last)
// Merges the adjacent subarrays a[first..mid] and a[mid + 1..last].
beginHalf1 = first endHalf1 = mid beginHalf2 = mid + 1  endHalf2 = last
//While both subarrays are not empty, compare an entry in one subarray with
// an entry in the other; then copy the smaller item into the temporary array
index = 0 //Next available location in tempArray
while ( (beginHalf1 <= endHalf1) and (beginHalf2 <= endHalf2) )
{
        if (a[beginHalf1] <= a[beginHalf2])
        {
                tempArray[index]  =  a[beginHalf1] beginHalf1++
        }
        else
        {
                tempArray[index]  =  a[beginHalf2] beginHalf2++
        }
        index++
}
// Assertion: One subarray has been completely copied to tempArray.
Copy remaining entries from other subarray to tempArray
Copy entries from tempArray to array a
```

Merge Sort

- FIGUR
during a



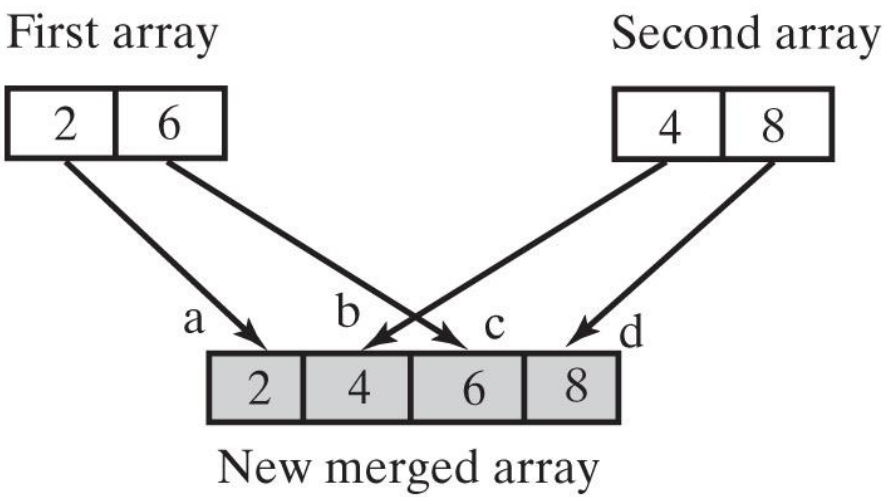
© 2019 Pearson Education, Inc.

Recursive Merge Sort

Be careful to allocate the temporary array only once.

```
public static <T extends Comparable<? super T>>
                        void mergeSort(T[] a, int first, int last)
{
    // The cast is safe because the new array contains null entries
    @SuppressWarnings("unchecked")
    T[] tempArray = (T[])new Comparable<?>[a.length]; // Unchecked cast
    mergeSort(a, tempArray, first, last);
} // end mergeSort
```

Merge Sort



© 2019 Pearson Education, Inc.

a. 2 < 4, so copy 2 to new array

b. 6 > 4, so copy 4 to new array

c. 6 < 8, so copy 6 to new array

d. Copy 8 to new array

Efficiency is O(n log n)

A worst-case merge of two sorted arrays

Iterative Merge Sort

- Less simple than recursive version.
  - Need to control the merges.
- Will be more efficient of both time and space.
  - But, trickier to code without error.

Iterative Merge Sort

- Starts at beginning of array
  - Merges pairs of individual entries to form two-entry subarrays
- Returns to the beginning of array and merges pairs of the two-entry subarrays to form four-entry subarrays
  - And so on
- After merging all pairs of subarrays of a particular length, might have entries left over.

Merge Sort in the Java Class Library

- Class **`Arrays`** in the package **`java.util`** defines versions of a static method sort

```
public static void sort(Object[] a)

public static void sort(Object[] a, int first, int after)
```

Comparing Algorithms

The time efficiencies of three sorting algorithms, expressed in Big Oh notation

	$O(n^2)$	$O(n^2)$	$O(n^2)$
	$O(n)$	$O(n^2)$	$O(n^2)$
	$O(n)$	$O(n^{1.5})$	$O(n^{1.5})$